

Введение в рекомендательные системы

ШАД, Рекомендательные системы
2025/26, лекция 1

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

На лекции:

1. Организационные вопросы
2. Информационная перегрузка
3. Что оптимизирует рекомендательная система
4. Как устроены рекомендательные алгоритмы
5. Зачем для рекомендаций нужны нейросети
6. Трансформеры для рекомендаций

На семинаре:

1. Поисследуем популярные датасеты
2. Построим первые рекомендации фильмов или аниме

Организационные вопросы

Часть 1



Команда курса



Даниил
Ткаченко



Кирилл
Хрыльченко



Роман
Нигматуллин



Артём
Матвеев



Владимир
Байкалов



Алексей
Красильников



Владислав
Тыцкий



Руслан
Кулиев



Валентина
Бронер

Кирилл Хрыльченко

Преподаватель и исследователь из
ФКН ВШЭ

- Окончил ВМК МГУ
- 5 лет занимался нейросетевыми рекомендательными системами в Яндексе
 - Руководил RecSys R&D-командой
- Первый автор статей на RecSys'25 и KDD'26
- Преподаю рекомендательные системы в ШАД и ВШЭ
- tg-канал @inforetriever



Цель курса

1. Передать вам наши знания
2. Научить вас:
 1. Строить рекомендательные системы
 2. Обучать нейросети для рекомендаций
 3. Заниматься R&D и исследованиями

Prerequisites:

1. Владеете Python и машинным обучением
2. Умеете обучать нейросети

Программа курса

Классическая часть курса:

1. Введение в рексистемы
2. Кандидатогенерация и метрики
3. Ранжирование и метрики
4. Хранение и обработка данных

Программа курса

Нейросетевая часть курса:

5. Нейросетевая генерация кандидатов

1. Архитектура
2. Обучение

6. Нейросетевое ранжирование

1. Обработка признаков
2. Архитектура

Программа курса

Прикладные и advanced темы:

7. Дизайн высоконагруженных рекомендательных систем
8. Рекомендательные трансформеры на практике
9. RL в рекомендациях
10. Case Studies рекомендательных продуктов
11. Тренды в RecSys

Домашние задания

1. Разработка мини-фреймворка рекомендаций (3 ДЗ)
2. Обучение нейросетевых моделей (3 ДЗ)
 1. Двухбашенные нейросети для генерации кандидатов
 2. Нейросетевое ранжирование
 3. Трансформеры для рекомендаций
3. Контест на Kaggle на основе датасета Yambda

Ресурсы: Google Collab, DataSphere, GPU-ресурсы ШАД

Финальная оценка

Финальная оценка = $0.25 * \text{ДЗ1} + 0.375 * \text{ДЗ2} + 0.375 * \text{Kaggle}$, где

- ДЗ1 – блок ДЗ про мини-фреймворк
- ДЗ2 – блок ДЗ про нейросети

Отлично – 8.5+, хорошо – 7.0+, удовлетворительно – 6.0+

Мягкие дедлайны: штраф $\min(10 * d, 50)\%$, где d – количество дней после дедлайна

Мидтерм: нужно набрать не менее 50% баллов за ДЗ1

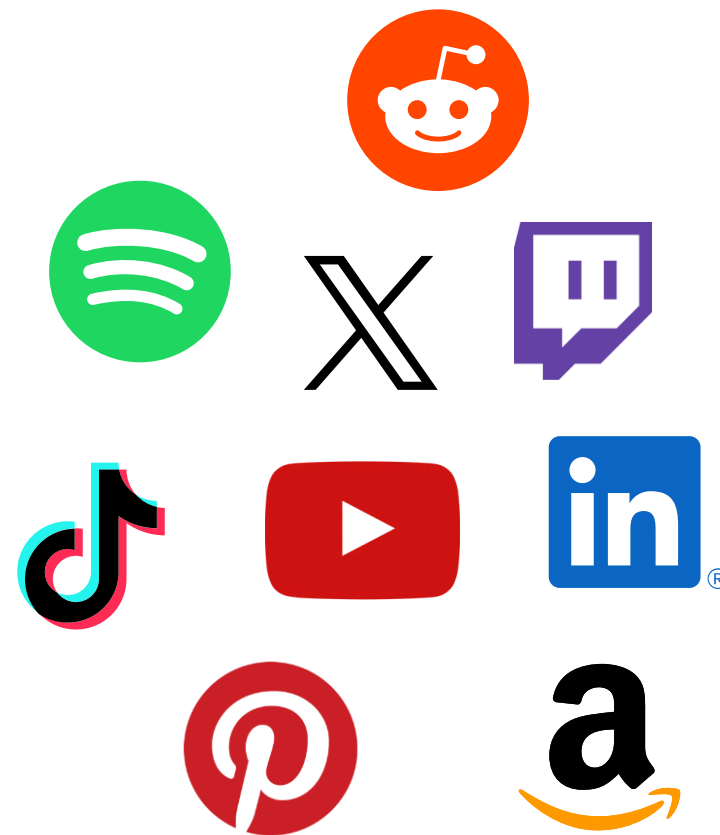
Информационная перегрузка

Часть 2



Цифровой мир = бесконечный каталог

- Музыка: >100 млн треков
- Видео: миллиарды роликов, десятки миллионов новых в день
- Социальные сети: сотни миллионов постов ежедневно
- Маркетплейсы: десятки-сотни миллионов товаров





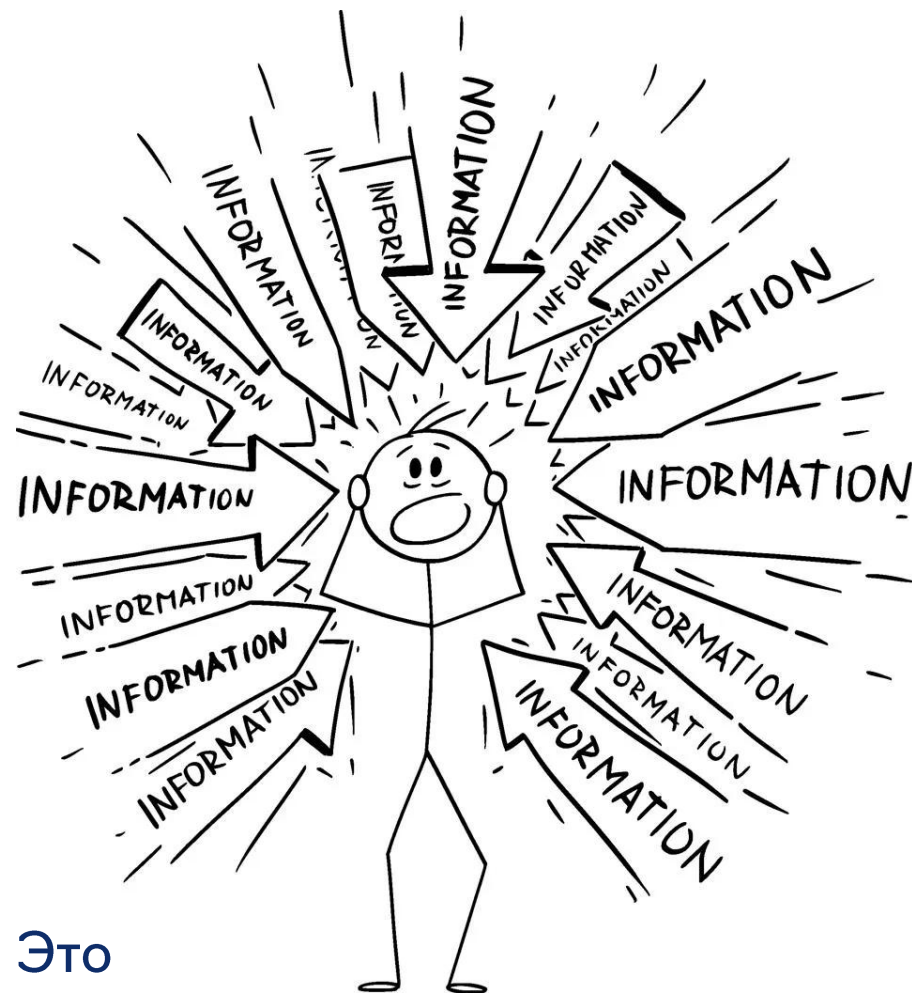
Масштабы YouTube

2022:

- 10 млрд видео
- 500 часов нового контента каждую минуту
- 1 млрд часов просмотров в день

Чтобы посмотреть всё – нужны десятки тысяч лет.

А ещё – 4% видео дают 94% всех просмотров.

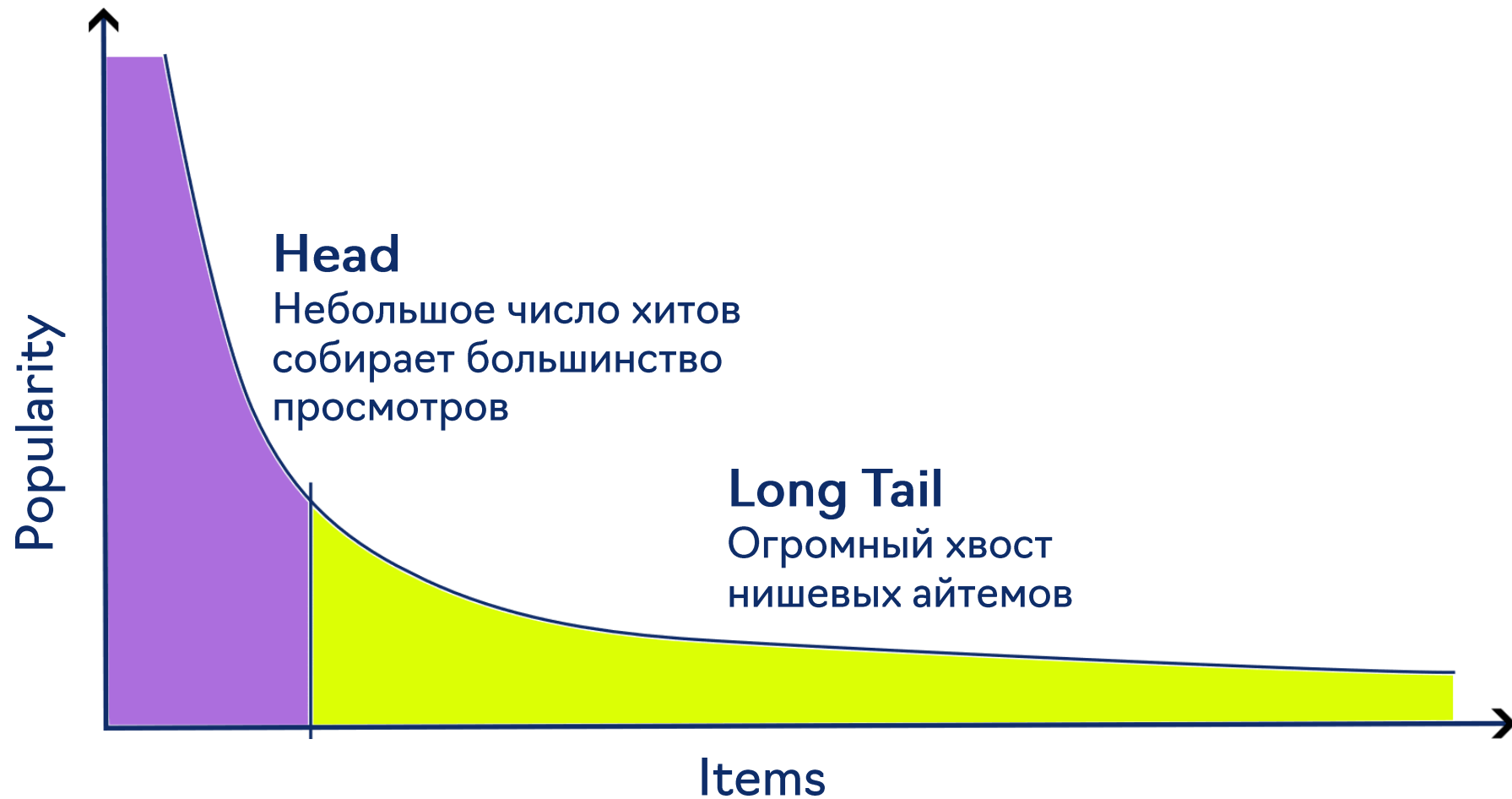


Это

**информационная
перегрузка**

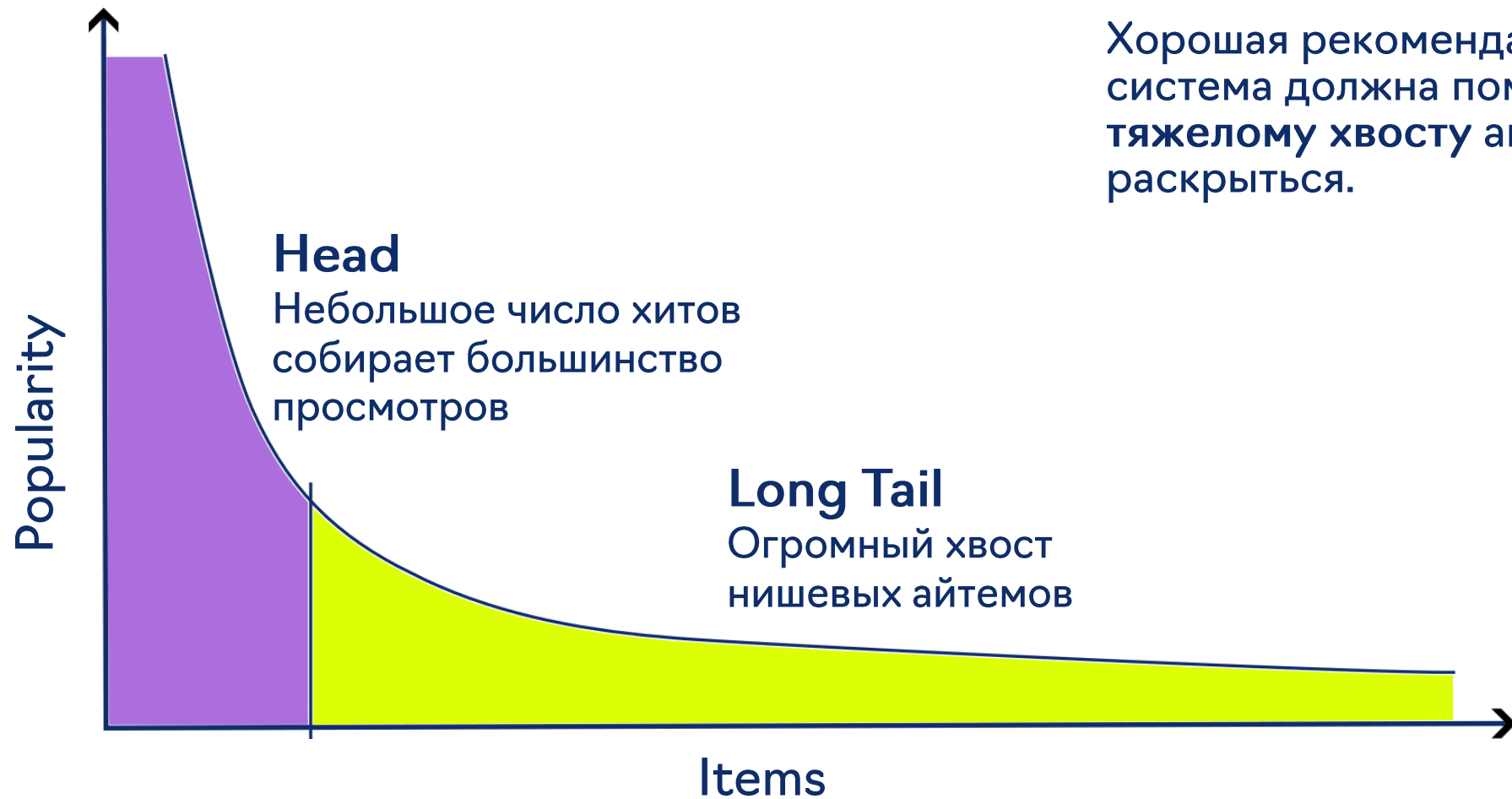
Тяжелый хвост айтемов

(элементы каталога,
которые мы рекомендуем)



Тяжелый хвост айтемов

(элементы каталога,
которые мы рекомендуем)



Хорошая рекомендательная
система должна помогать
тяжелому хвосту айтемов
раскрыться.

Возникновение рексистем

Рассмотрим три кейса:

- Netflix
- Социальные сети
- Spotify

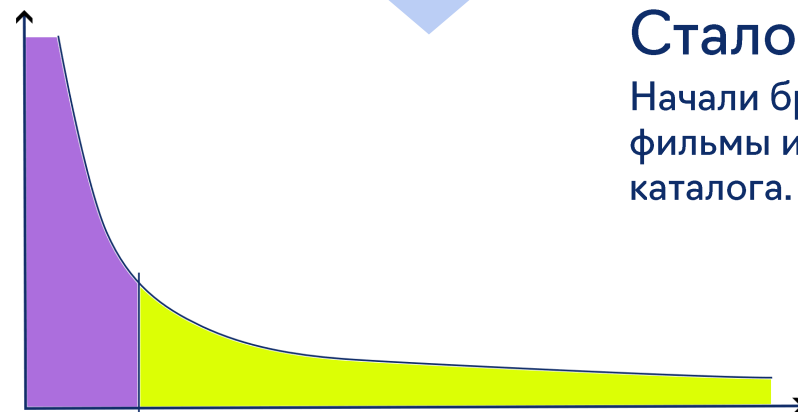
NETFLIX

RecSys 2014 Keynote by Neil Hunt: Quantifying the Value of Better Recommendations

Рассылка DVD по почте:



Добавили персонализированные рекомендации на основе пользовательского фидбека (5-star ratings).



Социальные сети

Сначала: News Feed = хронологическая лента постов друзей.

Позже – алгоритмическая лента:

- учитывается релевантность поста пользователю (поведение и контент)
- Свежесть тоже влияет



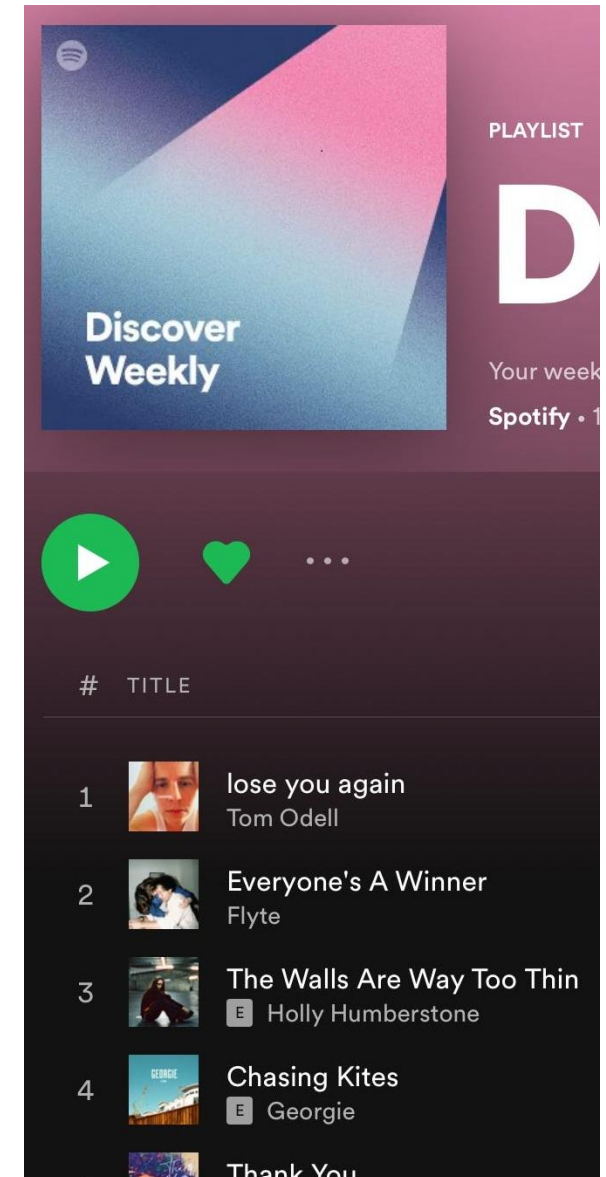
Это полноценная
**рекомендательная
система**



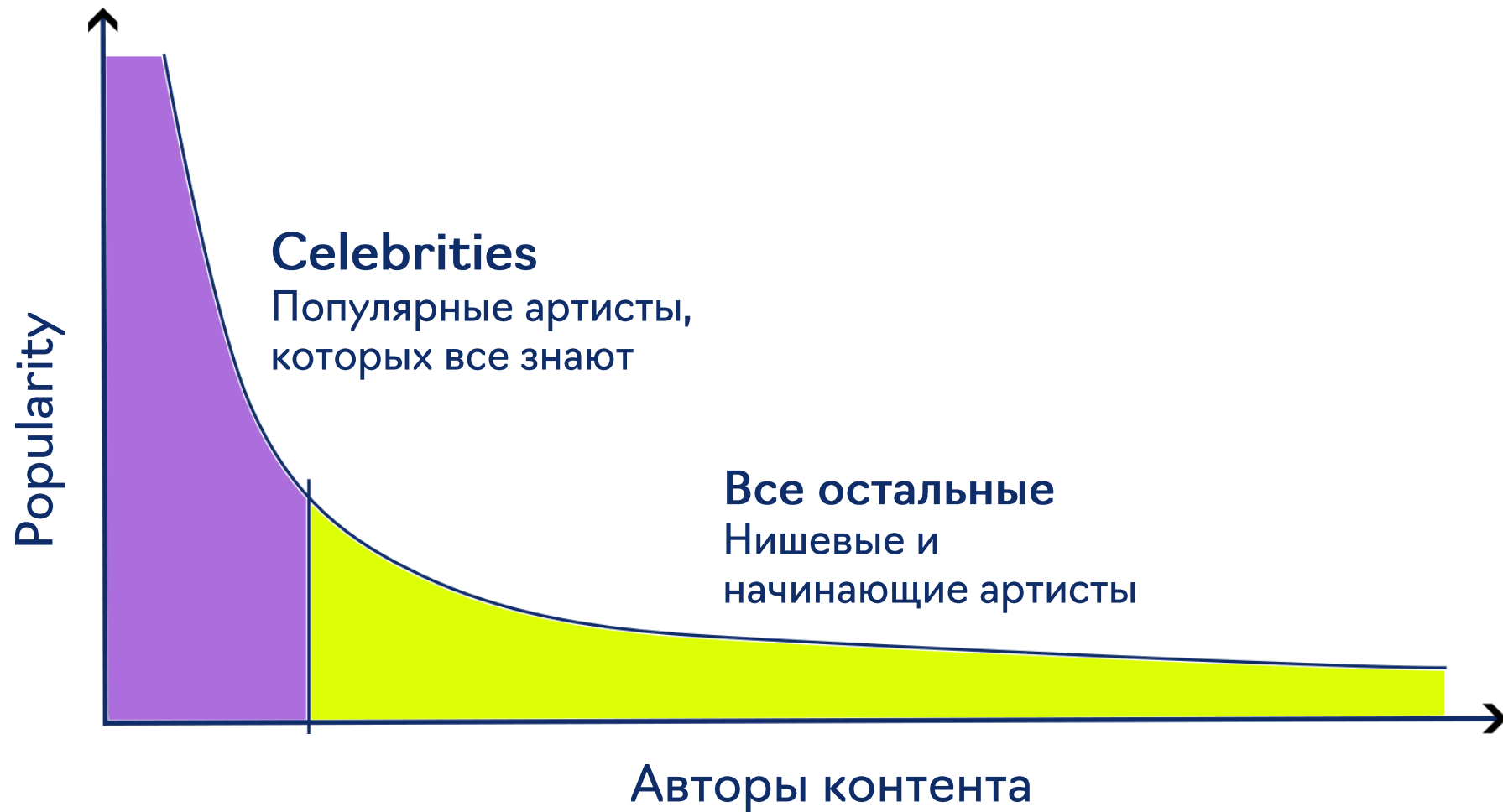
- Изначально (с 2008 года) – ручные плейлисты пользователей и редакторов
- В 2013 разработчик Erik Bernhardsson сколотил первую рекомендательную систему на основе матричной факторизации
- Через 2 года начали выпускать еженедельный плейлист **Discover Weekly**
- Через 3 месяца – 75 млн слушателей и 1.7 млрд прослушиваний



Discovery – помогаем пользователям находить новый контент, который они сами бы не нашли



Зачем рекомендательные системы авторам



Что оптимизирует рекомендательная система

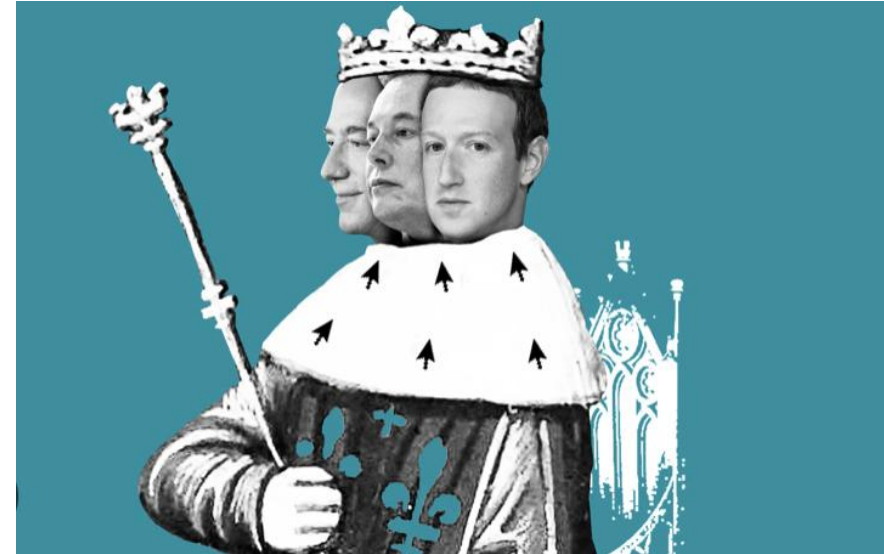
Часть 3



Платформа

Кроме пользователей и авторов контента, есть третья сторона (англ. stakeholder) – **платформа**:

- Разрабатывает рекомендательную систему
- Конечная цель – зарабатывать деньги (монетизация)



Технофеодализм:

- Онлайн-платформы – **феодалы**: контролируют цифровое пространство
- Пользователи и авторы – **крепостные**: предоставляют свои данные и труд в обмен на доступ к сервисам

Бизнес-модели платформ

Как зарабатывают:

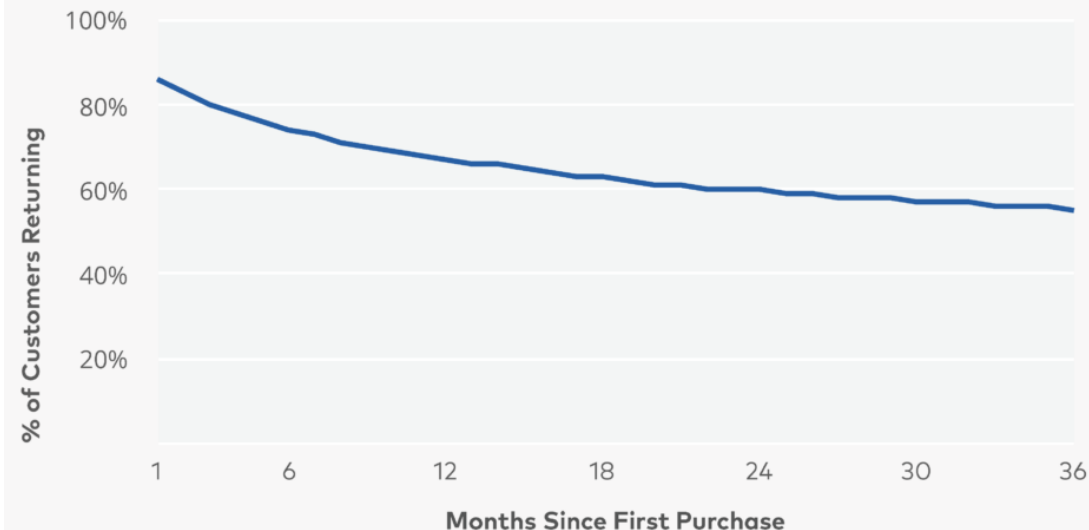
- Подписочные сервисы
- Маркетплейсы
- (spoiler)

Подписочные сервисы

Цель платформы – предоставлять достаточно ценности, чтобы:

- Удерживать текущих подписчиков (retention)
- Привлекать новых

Netflix - U.S. Customer Retention



* Mean retention across monthly cohorts from August 2014 to August 2019.

 Second Measure



Проблема холодного старта
(пользователей): трудно угадывать
интересы новых пользователей.

NETFLIX

300 млн платных
подписчиков



280 млн

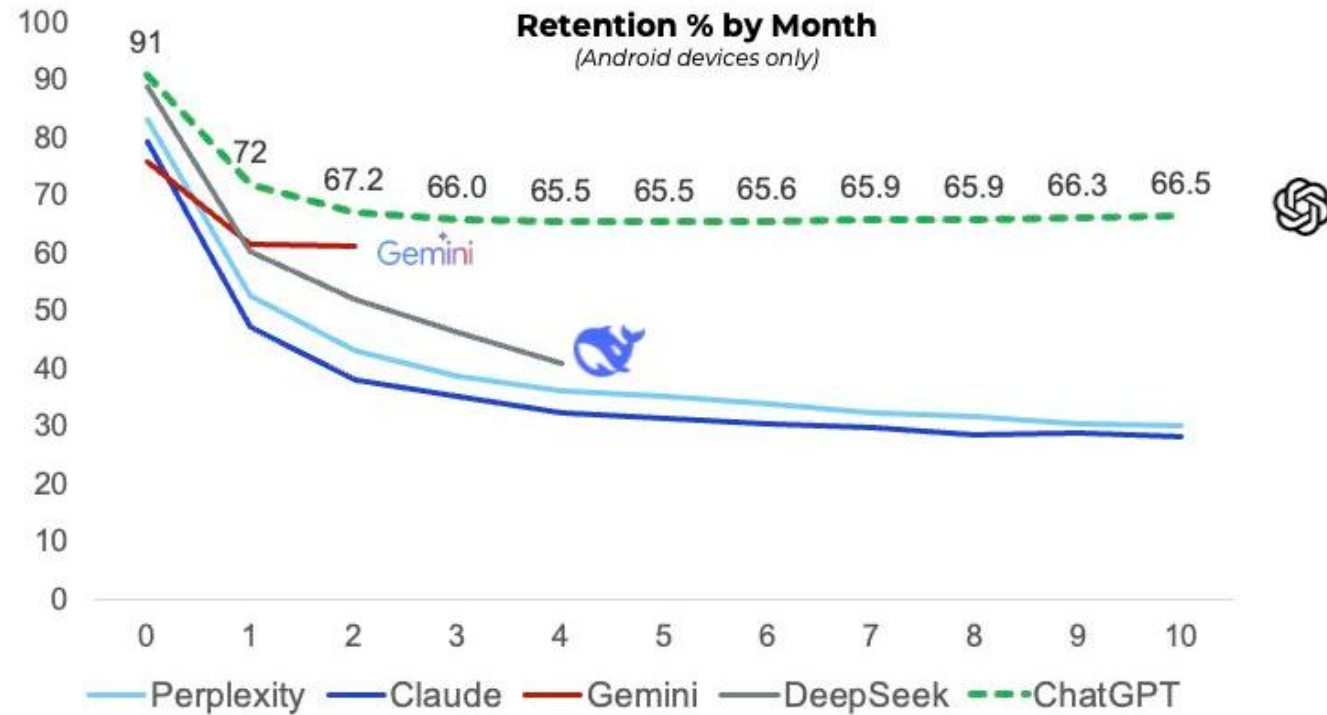


30 млн



10 млн

Retention Curve Smile



Source: Sensor Tower, Android only

Маркетплейсы

Платформа зарабатывает за счет комиссии с каждой продажи товара.

Цель платформы – максимизировать GMV:

- Вместо лучших рекомендаций выдаем те, которые в среднем приносят больше денег
- $\text{Score} = P(\text{покупка}) \times \text{цена}$

Динамическое ценообразование – один и тот же товар стоит по-разному в зависимости от контекста.



GMV (Gross Merchandise Volume) – общая стоимость товаров, проданных через платформу.

a

\$790 млрд в год (GMV)

Alibaba Group
阿里巴巴集团

Taobao: \$790 млрд
Tmall: \$683 млрд



\$554 млрд

wildberries

\$7.2 млрд

Largest Internet Companies

List of the largest Internet companies [hide]									
Rank ↕	Company ↕	Revenue USD billions ↕	F.Y. ↕	Employees ↕	Market cap. USD billions ↕	Headquarters ↕	Founded ↕	Industry ↕	Refs ↕
1	Amazon	\$620.12	2021	1,608,000	\$2,490	 Seattle	1994	Ecommerce	[1][2]
2	Alphabet	\$339.85	2021	156,500	\$2,270	 Mountain View	1998	Internet	[3][4]
3	JD.com	\$260.4	2021	385,357	\$390.35	 Beijing	1998	Ecommerce	[5][6]
4	Meta	\$156.22	2021	71,970	\$1,803	 Menlo Park	2004	Social Media	[7][8]
5	Alibaba	\$109.48	2021	251,462	\$330.67	 Hangzhou	1999	Ecommerce	[9][10]
6	Tencent	\$87.85	2021	112,771	\$562.84	 Shenzhen	1998	Internet	[11][12]
7	ByteDance	\$58	2021	110,000	\$353	 Beijing	2012	Social Media	[13][14]
8	Netflix	\$39	2021	12,135	\$434.46	 Los Gatos	1997	Entertainment	[15][16]
9	Meituan	\$27.77	2021	100,033	\$177.34	 Beijing	2010	Ecommerce	[17][18]
10	PayPal	\$25.37	2021	30,900	\$220.26	 San Jose	1998	Financial Services	[19][20]
11	Wildberries	\$22.26	2022	154,000	\$14.52	 Moscow	2004	Ecommerce	

Delightful Ads



in • 2nd

Senior Engineering Manager at Google Inc, improving YouTube ...

3mo • 

[+ Follow](#) 

We're looking for passionate ML Engineers to join our team and revolutionize how users discover delightful ads on YouTube, directly impacting the success of small and medium businesses!

Are you skilled in building recommendation models with hands-on experience in information retrieval, transformers, and LLMs? Do you thrive on challenging tasks that blend cutting-edge technology with real-world impact?

If you're ready to make a difference and help businesses thrive, we want to hear from you!

Apply here: <https://lnkd.in/ga3BAJUT> 

DM me if you are already past the interview loop.

[#MachineLearning](#) [#MLEngineer](#) [#YouTubeAds](#) [#Recommendations](#) [#Hiring](#)
[#CareerOpportunity](#) [#AI](#) [#LLM](#) [#Transformers](#)

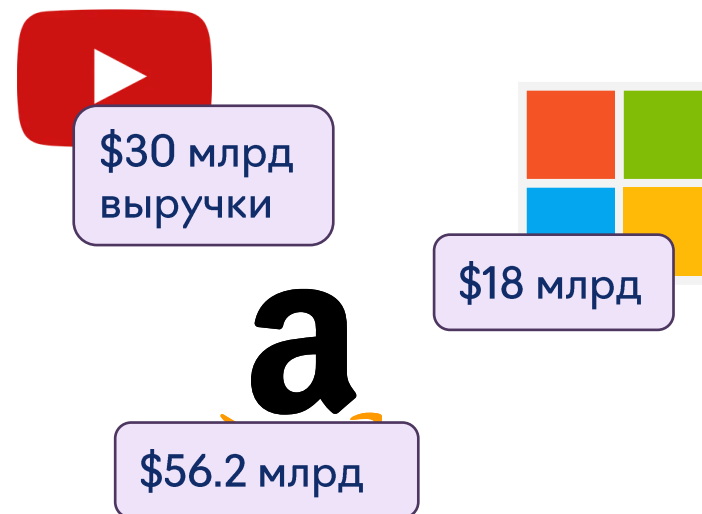
Монетизация через рекламу

YouTube:

- Во время просмотра видео, пользователю показывается реклама
- Чем больше watch time, тем больше показов рекламы

Больше пользовательской активности

→ больше рекламы → больше денег.



Монетизация поиска: показы рекламных документов рядом с "органикой".

Google Search: \$198 млрд выручки

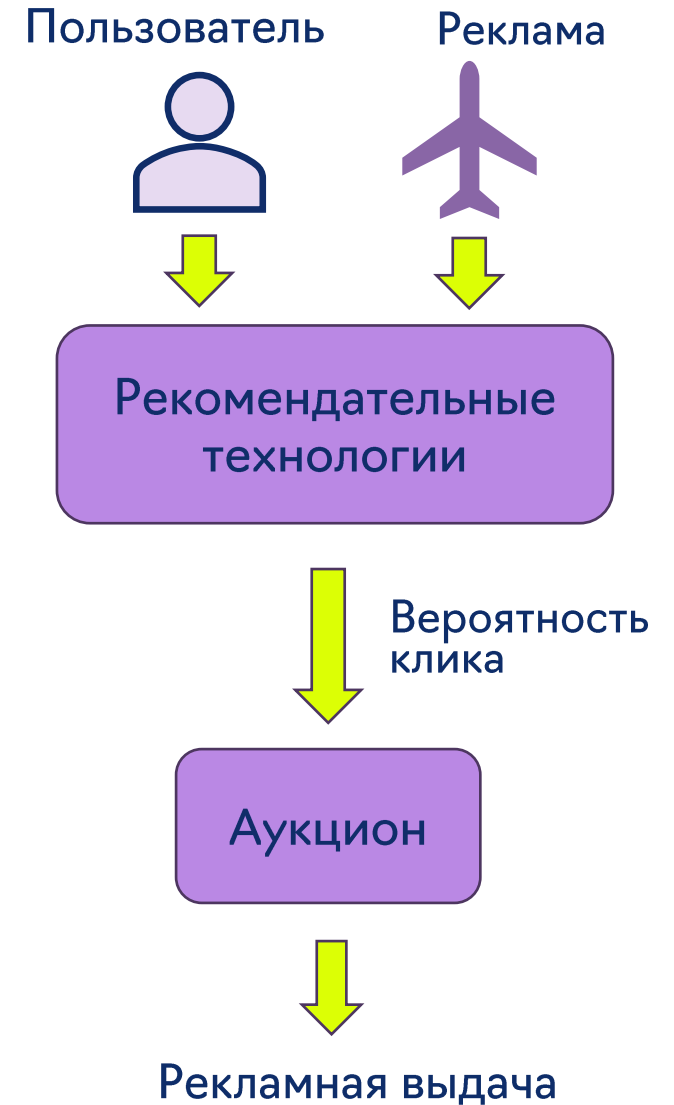
Персонализация рекламы

Рекламу показывают персонализированно:

- Чтобы меньше портить пользователю experience
- Чтобы она приносила больше пользы рекламодателям (и они продолжали за неё платить)

Для рекламы используются те же самые рексистемы, но:

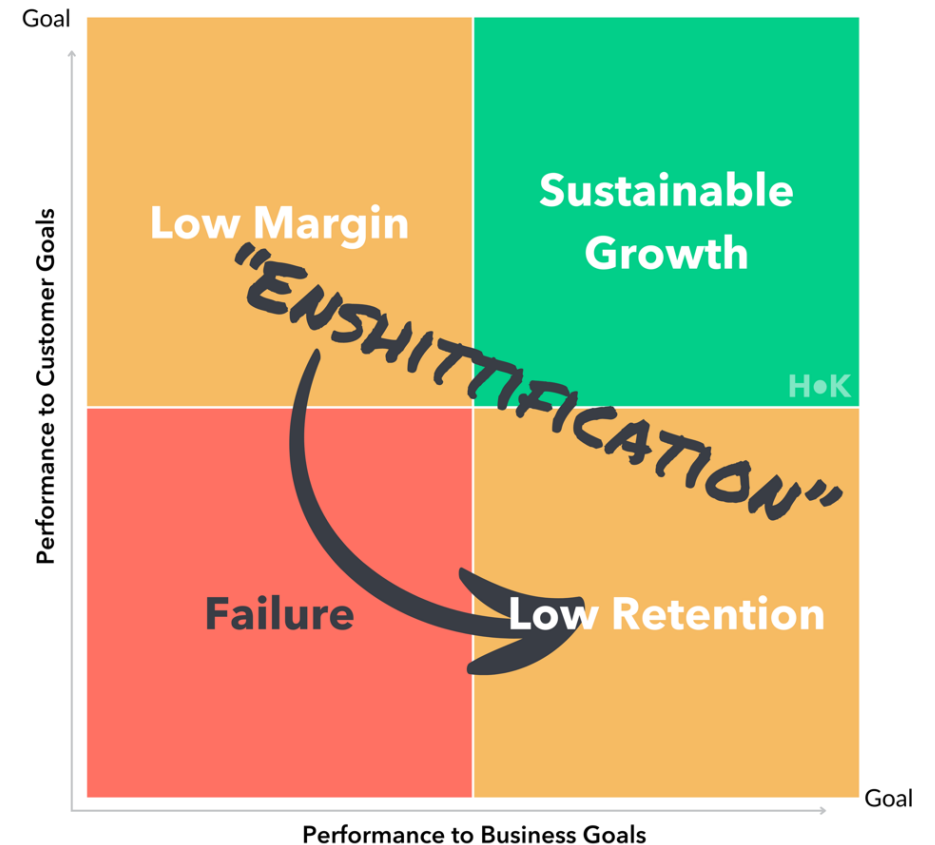
- Есть аукцион – учитываем сколько готов заплатить рекламодатель



Enshittification

Три стадии развития онлайн-платформ по Cory Doctorow:

- Изначально компания фокусируется на ценности для пользователей
- Затем фокус смещается на авторов / продавцов / рекламодателей
- Когда пользователям и авторам уже сложно сменить платформу, максимизируется прибыль для инвесторов



Google



Изначальный девиз Google:

- **Don't be evil**

После корпоративной реструктуризации:

- **Do the right thing**

Telegram



Free

Telegram is free forever. No ads.
No subscription fees.

Счастье пользователей и авторов как долгосрочная метрика успеха

Если приносить пользователям и авторам ценность, деньги придут сами. Вместо краткосрочной прибыли, нужно:

- максимизировать **счастье пользователей**
- давать шанс всем авторам быть рекомендованными, включая менее популярных и новых

Самое ценное у рексистемы – это пользователи и авторы. Их гораздо сложнее приобрести, чем обеспечить монетизацию.



Монетизация – **следствие** хорошего продукта, а не его причина.

Как устроены рекомендательные алгоритмы

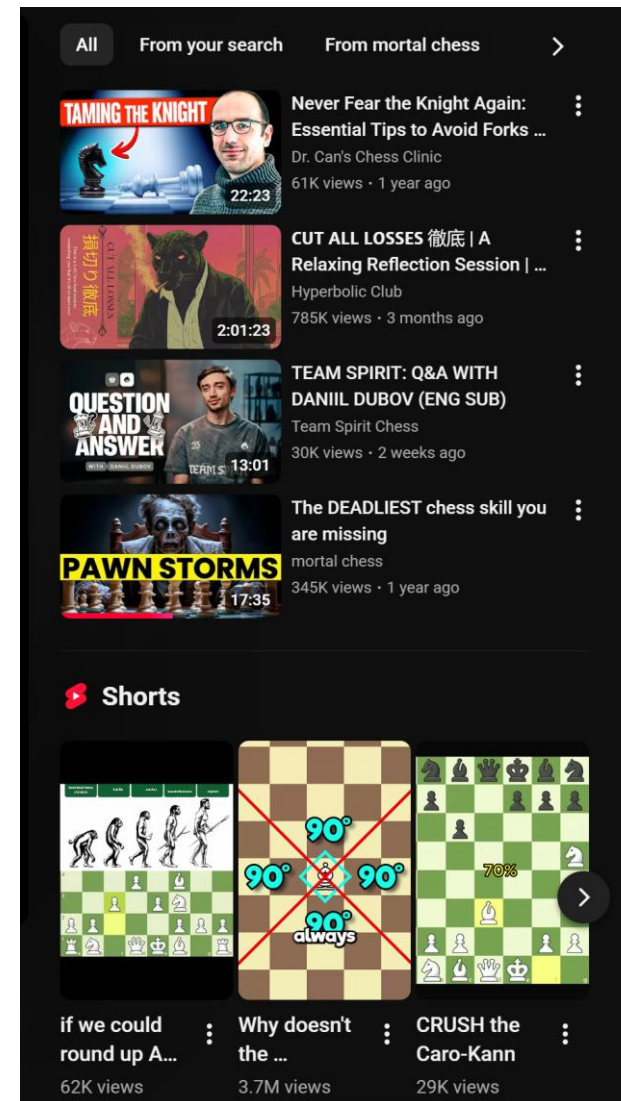
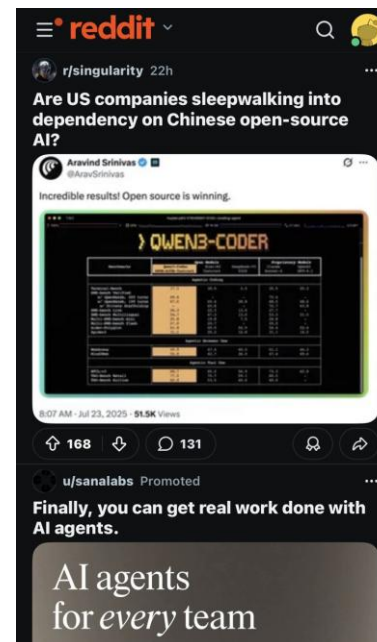
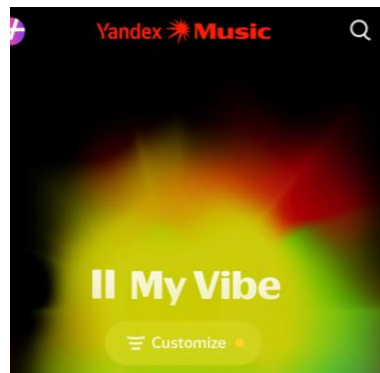
Часть 4



Результат работы рексистемы

- Следующий трек / видео / пост
- Лента новостей или рекомендаций
- Список похожих товаров
- Смесь разных форматов контента (и рекламы)

Интерфейсы разные, ранжирование одинаковое.



Рекомендательная система – это сортировка



- Рексистема считает функцию сортировки $f(u, c, i)$
- По значению f ранжирует весь каталог
- Вид f зависит от целей рексистемы:
 - Вероятность клика / лайка
 - Вероятность покупки, помноженная на цену
 - Ожидаемое время просмотра

Рекомендательная система – это сортировка



Контекст:

- Общий контекст – время, устройство, браузер, пользовательские настройки
- В рекомендациях похожих товаров – товар, на карточке которого находимся
- В поиске – поисковый запрос
- В рекламе – веб-страница, на которую зашёл пользователь

Скорость рекомендаций

Замедление на
100мс
→ -1% продаж

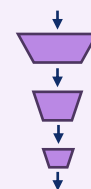
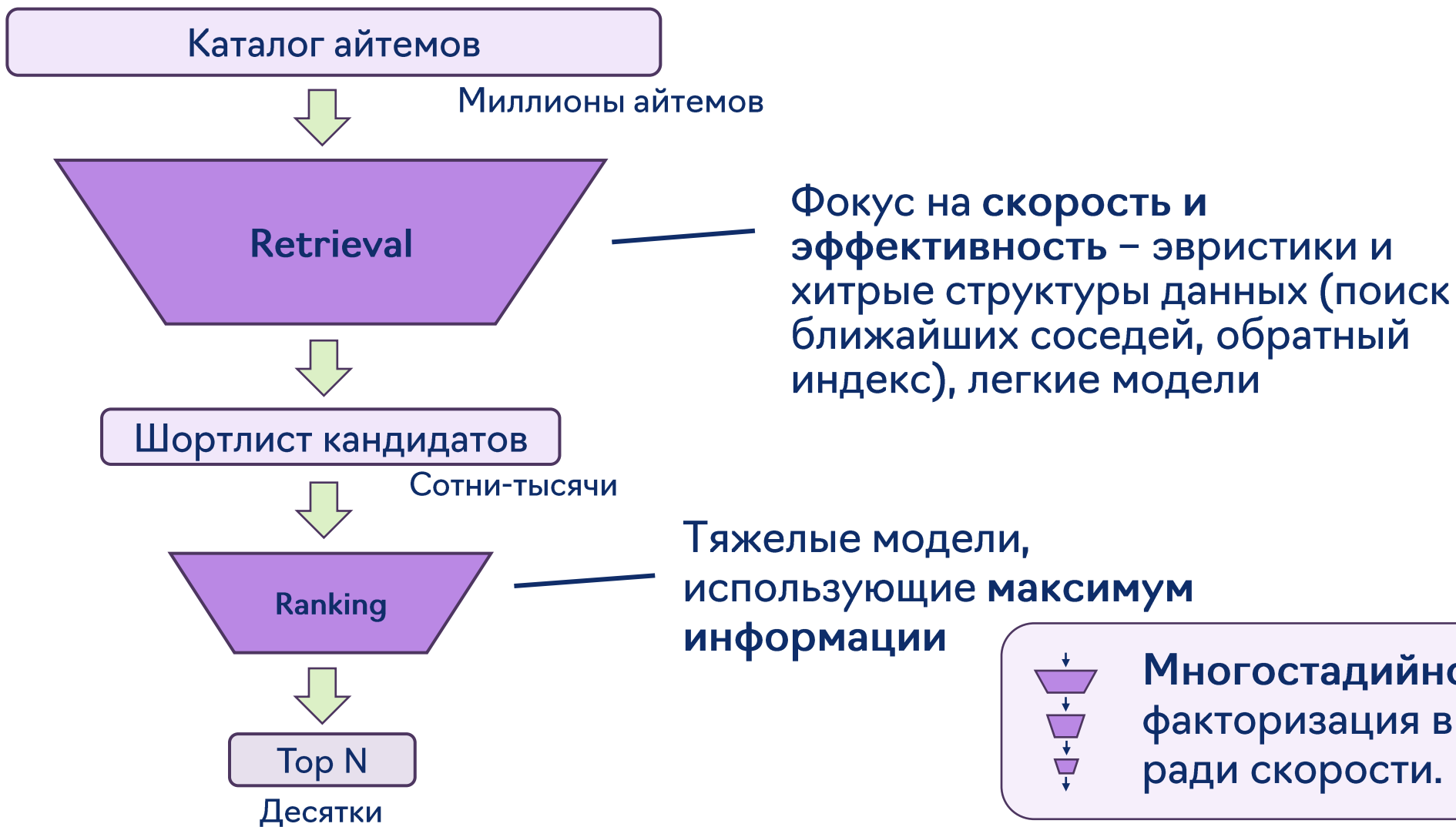


Замедление на
0.5сек
→ -20% трафика



Рекомендации должны формироваться за
десятки-сотни миллисекунд.

Многостадийные рексистемы



Многостадийность – это факторизация вычислений ради скорости.



Watch to watch next

Классический рекомендательный стек на примере YouTube до 2016 года – две стадии:

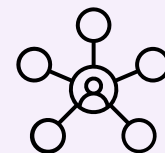
1. Генерация кандидатов: матричная факторизация
2. Ранжирование: линейная модель, предсказывающая expected watch time

Для того времени – это уже достаточно сложная рекомендательная система.

Коллаборативная фильтрация

Начнём с самых простых рекомендательных алгоритмов:

- Предлагаем пользователю то, что понравилось **похожим пользователям**
- В жизни: спрашиваем друзей, «что посмотреть/прочитать?»
- Рекомендательная система делает то же самое для миллионов пользователей

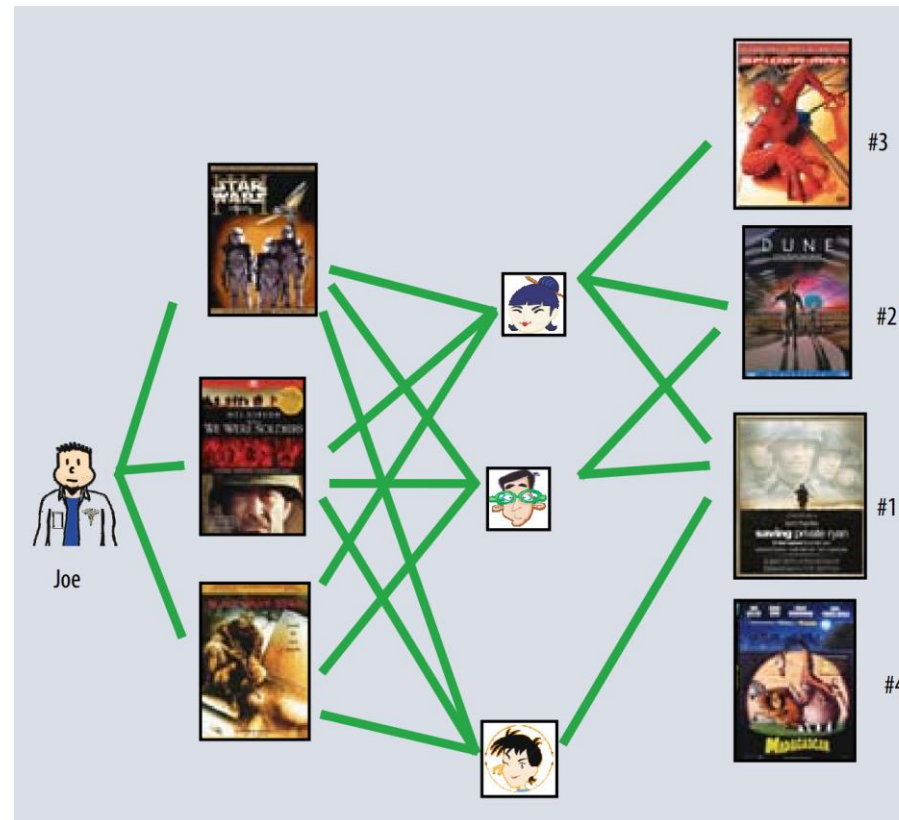


Коллаборативность – используем фидбек **всех** пользователей для рекомендаций **каждому**.

Как это работает (user-based CF)

- Смотрим, какие фильмы/песни/товары вам понравились
- Находим пользователей с похожими вкусами
- Берём то, что им понравилось, а вы ещё не видели
- Рекомендуем это вам

“Users like you also enjoyed X”



Matrix Factorization Techniques for Recommender Systems,
Yehuda Koren et al

Item-based CF: похожие айтемы

Можно смотреть не на похожих людей, а на похожие айтемы.

Как оценивать похожесть айтемов:

- По содержимому – описания, жанры, тэги
- По поведению пользователей (коллаборативный сигнал):
 - “80% слушавших трек X слушали и трек Y” → значит, X и Y похожи

Customers Who Bought This Item Also Bought



Холодный старт (айтемов):
сложно рекомендовать новинки,
потому что для них нет
пользовательского фидбека.

Как это видят алгоритмы: user-item матрица

- Строим большую матрицу релевантности R :
 - Строки – пользователи
 - Столбцы – айтемы
 - Элементы – user-item взаимодействия
- **User-based CF**: ищем похожие строки, переносим знания в пропуски
- **Item-based CF**: ищем похожие столбцы, используем их для рекомендаций

						Айтем
Пользователь	•	4	•	•	3	•
	•	•	2	3	1	•
	•	•	•	•	5	•
	1	2	•	•	•	•
	3	•	•	•	4	3
	•	2	•	5	1	•
	•	•	5	•	•	•

User-item interaction matrix

Разреженность

User-item матрица очень разреженная: 99+% элементов – нули.

Проблемы:

- Похожие пользователи могли ни разу не пересечься по лайкам → user-CF не видит их схожест
- «Rich gets richer, poor gets poorer»:
 - Популярные айтемы часто встречаются в user-item матрице → их много рекомендуем
 - Хвостовые айтемы почти не видим → системно их недооцениваем

NETFLIX

Netflix Prize:
99.82% sparsity



99.9999%
sparsity

Матричная факторизация

Представим user-item матрицу как произведение двух низкоранговых матриц:

$$\begin{bmatrix} \bullet & 4 & \bullet & \bullet & 3 \\ \bullet & \bullet & 2 & 3 & 1 \\ \bullet & \bullet & \bullet & \bullet & 5 \\ 1 & 2 & \bullet & \bullet & \bullet \\ 3 & \bullet & \bullet & \bullet & 4 \\ \bullet & 2 & \bullet & 5 & 1 \\ \bullet & \bullet & 5 & \bullet & \bullet \end{bmatrix} \approx \begin{bmatrix} 0.4 & 4.1 & 1.3 \\ 5.8 & 1.3 & 2.5 \\ 4.4 & 9.8 & 0.8 \\ 1.9 & 2.2 & 7.4 \\ 3.0 & 5.4 & 3.6 \\ 8.1 & 2.6 & 9.9 \\ 8.3 & 0.5 & 5.4 \end{bmatrix} \times \begin{bmatrix} 4.4 & 9.8 & 0.8 & 5.4 & 3.6 \\ 1.9 & 2.2 & 7.4 & 2.6 & 9.9 \\ 3.0 & 5.4 & 3.6 & 0.5 & 5.4 \end{bmatrix}$$

User-item interaction matrix User Embeddings Item Embeddings

Матричная факторизация

Представим user-item матрицу как произведение двух низкоранговых матриц:

The diagram illustrates the matrix factorization of a user-item interaction matrix. On the left is the 'User-item interaction matrix' (6x5), where dots represent missing values. A red box highlights the first two rows and the fourth column. This matrix is approximately equal to the product of two matrices. The first matrix is 'User Embeddings' (6x3), and the second is 'Item Embeddings' (3x5). A large 'X' symbol indicates the matrix multiplication.

•	4	•	•	3
•	•	2	3	1
•	•	•	•	5
1	2	•	•	•
3	•	•	•	4
•	2	•	5	1
•	•	5	•	•

≈

0.4	4.1	1.3
5.8	1.3	2.5
4.4	9.8	0.8
1.9	2.2	7.4
3.0	5.4	3.6
8.1	2.6	9.9
8.3	0.5	5.4

×

4.4	9.8	0.8	5.4	3.6
1.9	2.2	7.4	2.6	9.9
3.0	5.4	3.6	0.5	5.4

User-item interaction matrix User Embeddings Item Embeddings

Матричная факторизация

Представим user-item матрицу как произведение двух низкоранговых матриц:

The diagram illustrates the matrix factorization of a user-item interaction matrix. It shows three matrices: the User-item interaction matrix, the User Embeddings matrix, and the Item Embeddings matrix. The interaction matrix is approximated by the product of the two embedding matrices, as indicated by the approximation symbol ($\approx$) and the multiplication symbol ($\times$).

User-item interaction matrix

•	4	•	•	3
•	•	2	3	1
•	•	•	•	5
1	2	•	•	•
3	•	•	•	4
•	2	•	5	1
•	•	5	•	•

User Embeddings

0.4	4.1	1.3
5.8	1.3	2.5
4.4	9.8	0.8
1.9	2.2	7.4
3.0	5.4	3.6
8.1	2.6	9.9
8.3	0.5	5.4

Item Embeddings

4.4	9.8	0.8	5.4	3.6
1.9	2.2	7.4	2.6	9.9
3.0	5.4	3.6	0.5	5.4

Матричная факторизация

Представим user-item матрицу как произведение двух низкоранговых матриц:

The diagram illustrates the matrix factorization of a user-item interaction matrix into two low-rank matrices: User Embeddings and Item Embeddings. The interaction matrix is shown on the left, followed by an approximation symbol (≈), then the User Embeddings matrix, a multiplication symbol (×), and finally the Item Embeddings matrix.

User-item interaction matrix

•	4	•	•	3
•	•	2	3	1
•	•	•	•	5
1	2	•	•	•
3	•	•	•	4
•	2	•	5	1
•	•	5	•	•

User Embeddings

0.4	4.1	1.3
5.8	1.3	2.5
4.4	9.8	0.8
1.9	2.2	7.4
3.0	5.4	3.6
8.1	2.6	9.9
8.3	0.5	5.4

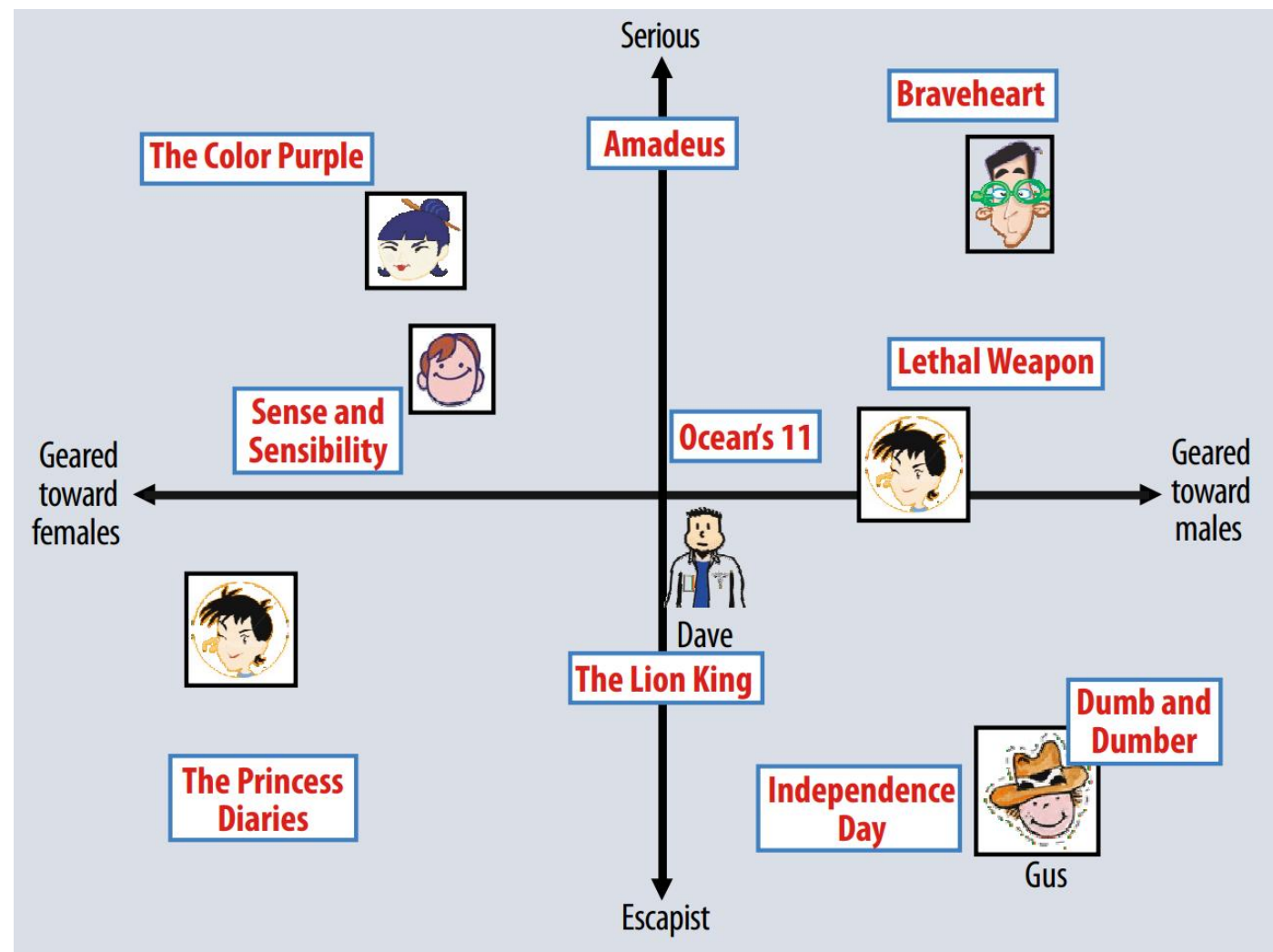
Item Embeddings

4.4	9.8	0.8	5.4	3.6
1.9	2.2	7.4	2.6	9.9
3.0	5.4	3.6	0.5	5.4

Интерпретируемость матричной факторизации

Каждый элемент векторного представления отражает какую-то характеристику:

- Например, пользователь любит ужастики
- Для айтема – та же координата подсвечивает, является ли он ужасником
- Элементы “совпадают” – высокая оценка



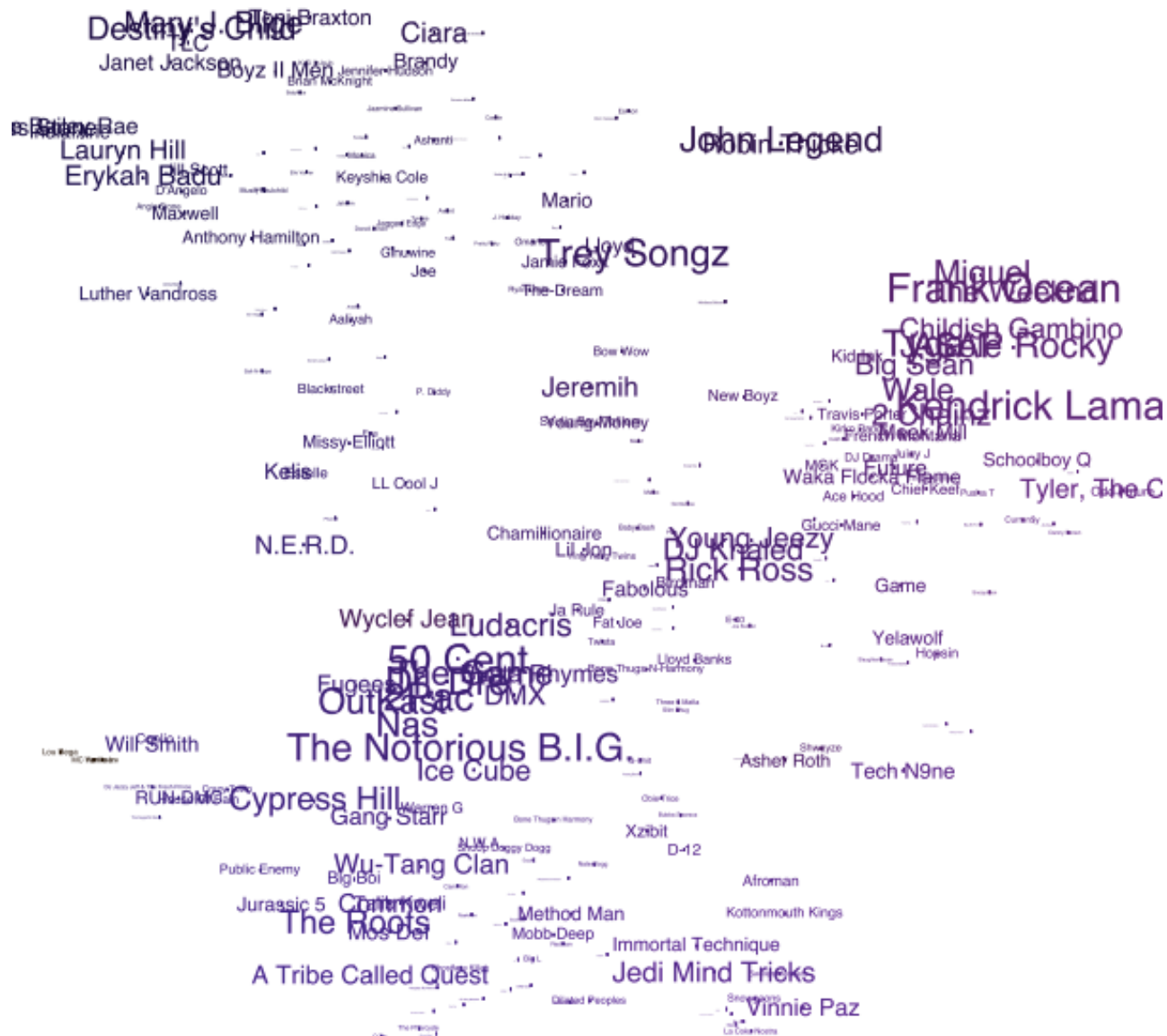


Вспомним Эрика из Spotify:

- Построил user-item матрицу прослушиваний треков
- С помощью матричной факторизации получил векторы длины 40



Нір-нор кластер



Рекомендации с помощью матричной факторизации

- Для всех пользователей и айтеров есть эмбединги (векторные представления), полученные из матричной факторизации
- Можем найти для пользователя айтеры с максимальным значением скалярного произведения (MIPS – maximum inner product search)
- Выдать их в качестве рекомендаций или заиспользовать как кандидатов для следующий стадий “рекомендательной воронки”

Преимущества матричной факторизации

- Побороли разреженность – у пользователей не пересекавшихся по айтемам могут быть похожие векторы
- Тяжелому хвосту стало лучше
- Научились находить абстрактные “вкусы” пользователей и характеристики айтемов
- Сжали информацию из огромной user-item матрицы в две небольших матрицы:
 - Описываем объекты небольшим количеством чисел вместо широких разреженных векторов
 - Подход хорошо масштабируется

Netflix Prize

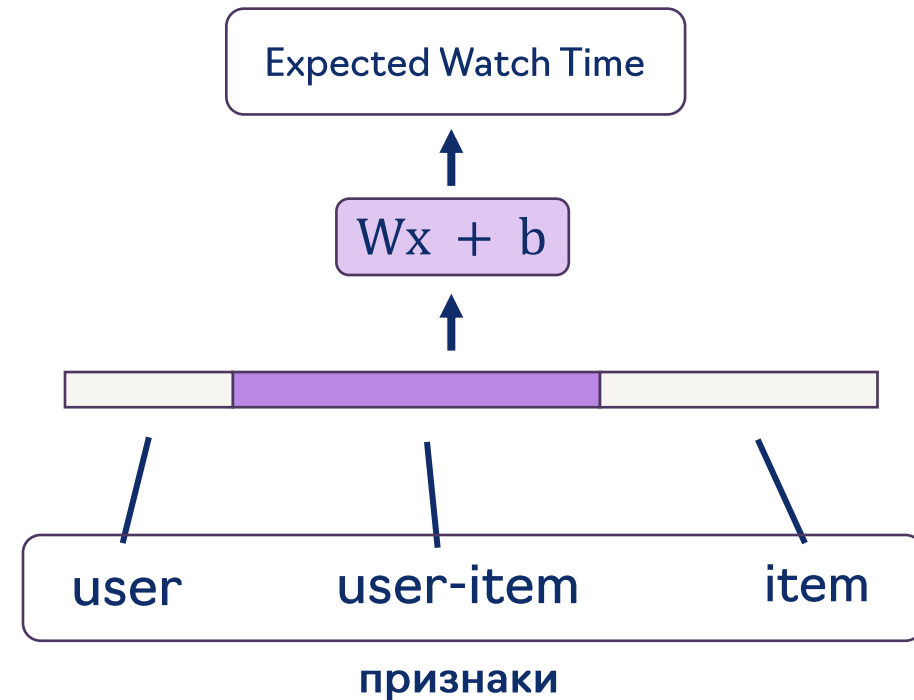
- В 2006 году Netflix опубликовали датасет с 100 млн проставленных фильмах оценок;
 - 480 тысяч клиентов, 17 тысяч фильмов
- Цель соревнования: улучшить точность относительно алгоритма Netflix на 10%
- Приз: **1 млн долларов**
- Матричная факторизация – ключевой подход для победы



№	Команда	СКО	Улучшение в %	Время отправки
1	The Ensemble	0.8553	10.10	2009-07-26 18:38:22
2	BellKor's Pragmatic Chaos	0.8554	10.09	2009-07-26 18:18:28

Линейная модель ранжирования

- Цель: показывать видео, которые будут смотреть дольше
- Модель предсказывает **expected watch time**
- **Признаковое пространство:**
 - признаки пользователя
 - признаки видео
 - user-item признаки (например, наличие подписки на автора)



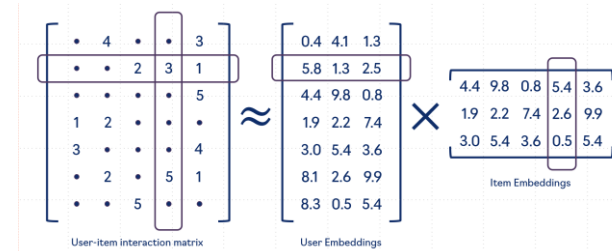


Watch to watch next

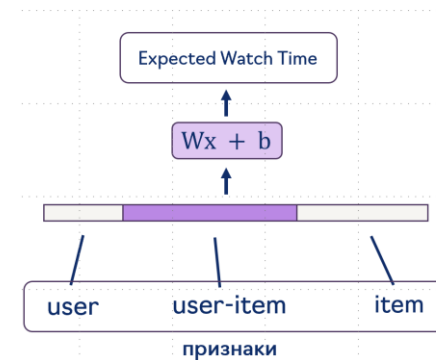
Классический рекомендательный стек на примере YouTube до 2016 года – две стадии:

1. Генерация кандидатов: матричная факторизация
2. Ранжирование: линейная модель, предсказывающая expected watch time

Отбираем тысячи кандидатов с помощью матричной факторизации



Ранжируем их с помощью линейной модели и берем top 10



Зачем для рекомендаций нужны нейросети

Часть 5



Зачем нужны нейросети

- У нас очень много данных – **триллионы user-item взаимодействий**
 - Классическому ML столько не нужно
- В других областях DL победил
- В рекомендациях есть свои челленджи:
 - холодный старт и тяжелые хвосты
 - distribution drift

Bitter lesson (Richard Sutton)



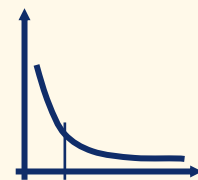
"The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin."



Scaling hypothesis – чем больше данных и сама модель, тем лучше результат.

Холодный старт и тяжёлый хвост

- В новостях, рекламе, маркетплейсах каталог постоянно обновляется:
 - для новых айтемов нет фидбека
- Нужен content understanding:
 - видео/текст/картинка → эмбединг
- Тяжелый хвост:
 - популярные айтемы “забивают” обучение
 - у контентных признаков хвост легче, больше шанс вытянуть объекты из хвоста



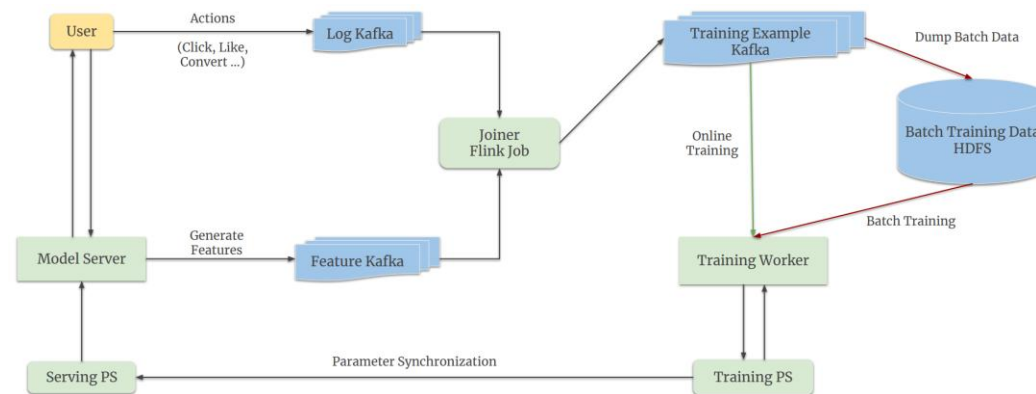
Matthew Effect: “the rich get richer and the poor get poorer.”

Он же popularity bias.

Distribution Drift

- Распределения постоянно меняются:
 - появляются новые тренды, обновляется каталог
 - меняются интересы пользователей
- Нужны алгоритмы, которые постоянно дообучаются на новых данных

Онлайн-дообучение нейросетевых рекомендаций TikTok



Monolith: Real Time Recommendation System With Collisionless Embedding Table

Feedback Loop



- Модель обучается на данных, которые породила
- Данные отражают её плохие особенности
- Обучение на этих данных эти особенности усиливает

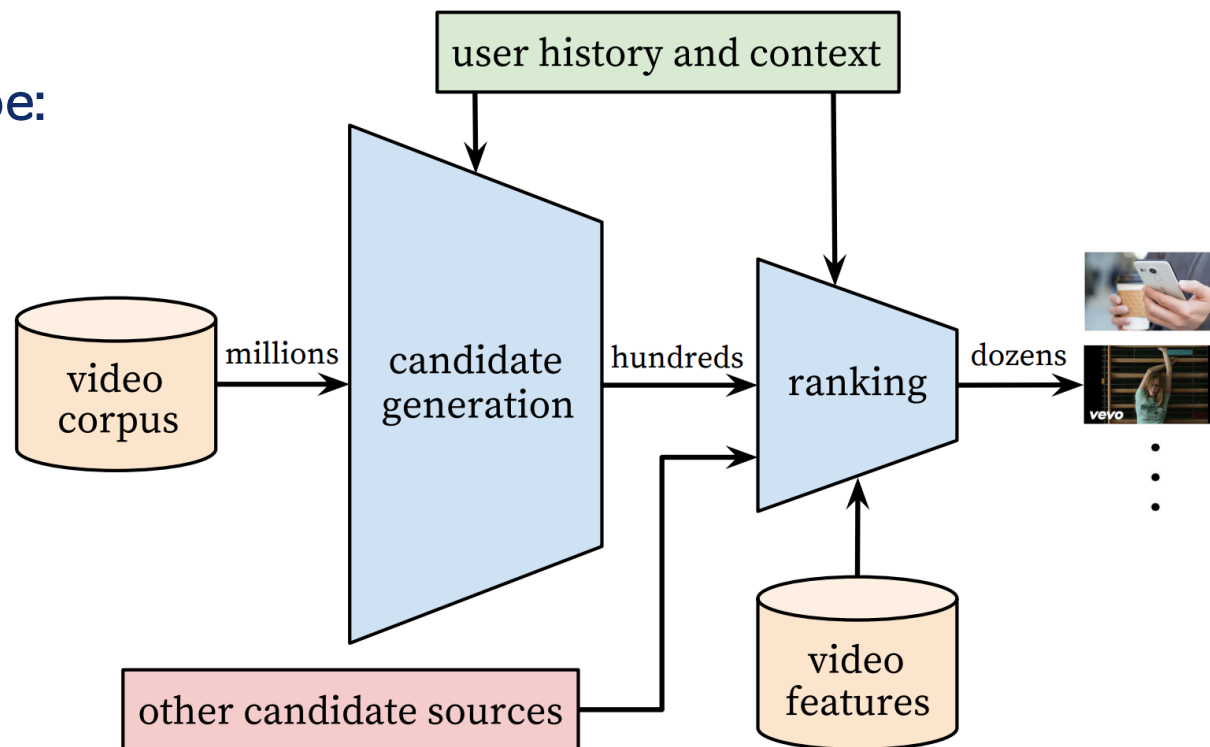
YoutubeDNN

В 2016 году Google представили нейросетевые рекомендации в Youtube:

- Двухбашенная нейросеть для генерации кандидатов
- Полносвязная нейросеть для ранжирования

До этого:

- Матричная факторизация для генерации кандидатов
- Линейная модель для ранжирования

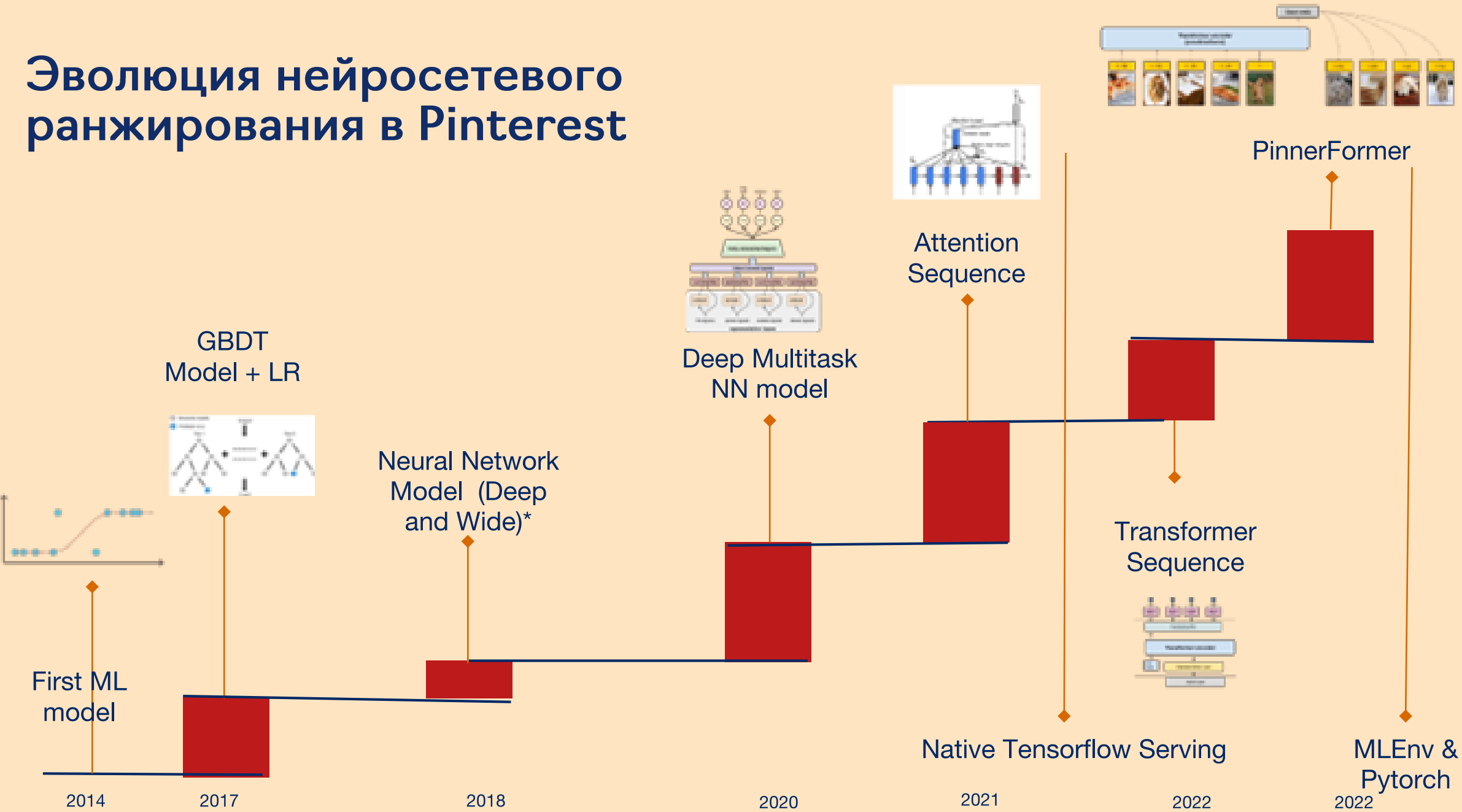


Deep Neural Networks for YouTube Recommendations

Ветки развития рексистемы

- Улучшение моделей для генерации кандидатов
- Улучшение ранжирующих моделей
- Концептуально другие подходы
 - делаем больше или меньше стадий в рексистеме
 - меняем интерфейс рекомендаций
- Про всё это поговорим в рамках курса

Эволюция нейросетевого ранжирования в Pinterest



Трансформеры для рекомендаций

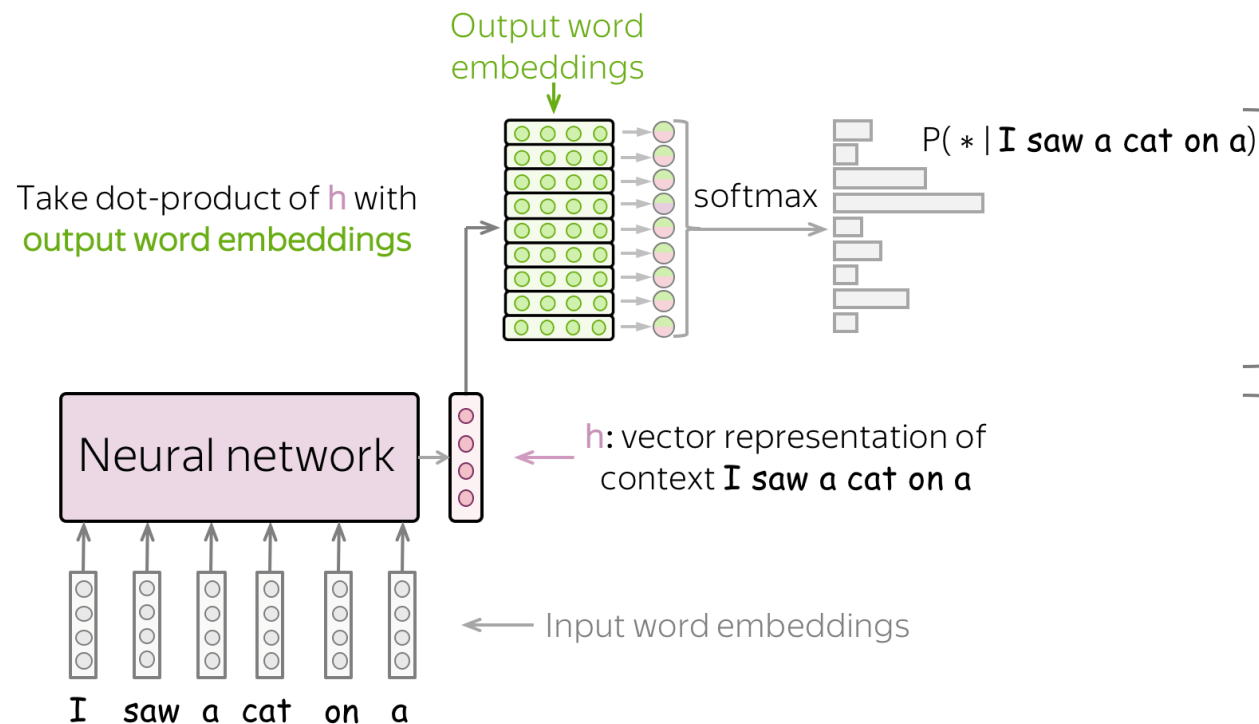
Часть 6



Языковые модели

ChatGPT, Gemini и другие большие языковые модели учатся **предсказывать следующее слово в тексте:**

- Extreme multi-task learning: учим грамматику, логику, факты, здравый смысл, даже математику и рекомендации



Из Lena Voita's [YSDA NLP Course](#)



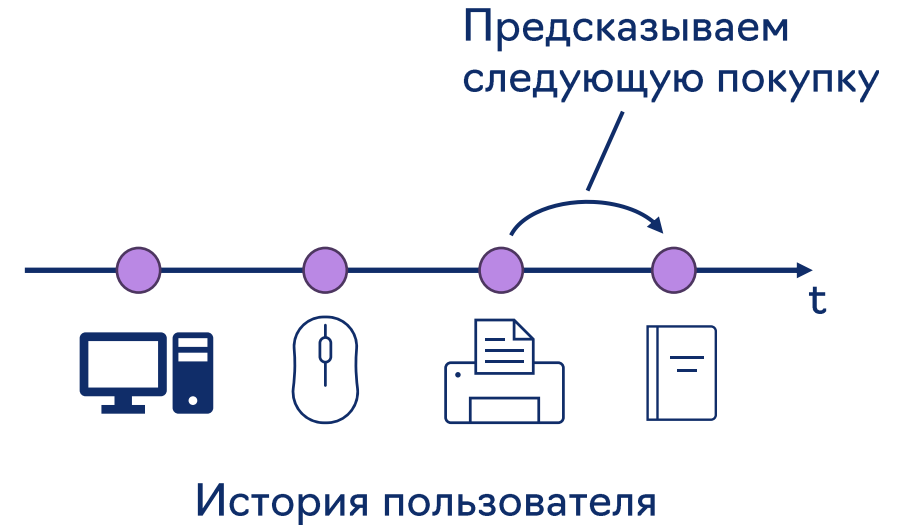
LLM – это модель мира: она учится понимать мир, продолжая тексты.

Next Item Prediction

Заменим слова на действия пользователей:

- было (NLP): “Я пошёл в [token1] [token2] ... “
- стало (RecSys): “Пользователю понравились [track1] [track2] ...”

С помощью задачи
предсказания следующего айтема
обучаем генеративную модель пользователя.



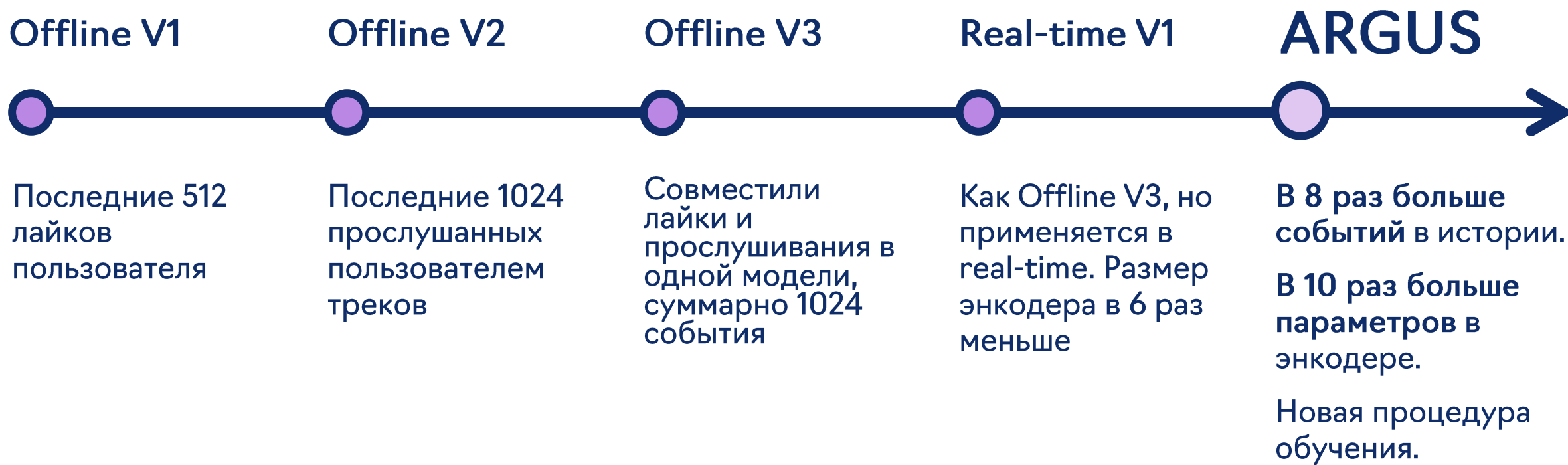
В отличие от NLP, пространство токенов очень большое:

- NLP: $10^4 - 10^5$, RecSys: $10^6 - 10^9$ токенов
- больше похоже на генерацию следующего кадра видео, чем на текст



Это модель мира! В которой мир – это пользователь.

Трансформеры в Яндекс Музыке



Уже есть и новые модели, смотрите [выступление Петра Зайделя на PML Conf.](#)

Трансформеры в Яндекс Музыке

Внедрение	Длина истории	Конфигурация	Параметры	TLT	Like Likelihood
Offline V1	512	L6 H512	18.9M	+0.52%	+1.11%
Offline V2	1024	L6 H512	18.9M	+1.00%	+0.73%
Offline V3	1024	L6 H512	18.9M	+0.73%	+5.00%
Real-time V1	1024	L4 H256	3.2M	+0.32%	+1.38%
Offline V4 (ARGUS)	8192	L10 H1024	126.0M	+2.26%	+6.37%

Summary

Обсудили:

1. Тяжелые хвосты и возникновение рекомендаций в Netflix, Spotify, соцсетях
2. Подписочные сервисы, маркетплейсы, реклама
3. Многостадийность, коллаборативную фильтрацию, матричную факторизацию, линейные модели
4. Зачем нужны нейросети для рекомендаций
5. Трансформеры